

Table of Contents

Habesha Market — Project Documentation

Course: Web Programming

Project Title: Habesha Market — Ethiopian Online Marketplace

Technologies: HTML5, CSS3, JavaScript (Vanilla), PHP (MySQLi), MySQL

Project Weight: 20%

Table of Contents

1. System Overview and Objectives
 2. Project Architecture
 3. Database Schema and Entity Relationship Diagram
 4. File Structure
 5. Page-by-Page Code Analysis
 - 5.1 Shared Components (Header, Footer, Connection)
 - 5.2 Home Page (index.php)
 - 5.3 Products Listing (products.php)
 - 5.4 Product Detail (product_detail.php)
 - 5.5 Shopping Cart (cart.php)
 - 5.6 Checkout (checkout.php)
 - 5.7 Authentication (login.php / signup.php)
 - 5.8 User Profile (profile.php)
 - 5.9 Orders (orders.php / order_success.php)
 - 5.10 Admin Panel
 6. CSS Design System and Styling Conventions
 7. JavaScript Interactivity and AJAX Features
 8. AJAX Endpoints
 9. Security Implementation
 10. Setup Instructions
 11. Features Implemented
 12. List of Features vs. Course Requirements
-

1. System Overview and Objectives

Habesha Market is a fully functional dynamic e-commerce web application built to simulate a real Ethiopian online marketplace. It connects buyers with authentic Ethiopian products —

including coffee, traditional fashion (Habesha Kemis), spices, handcrafts, jewelry, and books — and allows users to browse, purchase, and track orders.

Primary Objectives:

- Provide an intuitive, mobile-responsive storefront for Ethiopian products
 - Enable secure user registration, login, and profile management
 - Implement a full product catalog with category filtering, live search, and sorting
 - Support a complete shopping flow: browse → add to cart → checkout → order confirmation
 - Offer an admin panel for managing products, viewing orders, and monitoring users
 - Demonstrate AJAX-powered interactivity including live search, cart updates, and email validation
-

2. Project Architecture

The application follows a traditional MVC-like structure using vanilla PHP, with no heavy frameworks. The project is organized into logical folders:

Layer	Technology	Role
Presentation	HTML5 + CSS3	Page structure and styling
Client Logic	Vanilla JavaScript	DOM manipulation, AJAX, validation
Server Logic	PHP (MySQLi)	Session handling, business logic
Data	MySQL	Persistent storage, CRUD operations

Session Management: PHP native sessions store user authentication state (user_id, full_name, email, role). Admin pages check `$_SESSION['role'] === 'admin'` before rendering.

Request Flow:

1. Browser requests a .php page
 2. PHP starts the session and checks authentication where required
 3. Page includes header.php (HTML head + navbar) and footer.php (footer + script tags)
 4. PHP queries MySQL via connection.php and renders dynamic HTML
 5. JavaScript in main.js attaches event listeners and handles AJAX calls post-render
-

3. Database Schema

The database habesha_market uses utf8mb4 encoding to support Amharic and other Ethiopian scripts.

Table: users

Stores registered accounts. Passwords are stored using PHP's password_hash() with PASSWORD_DEFAULT (bcrypt).

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique user identifier
full_name	VARCHAR(150)	User's full name
email	VARCHAR(200) UNIQUE	Login email (unique constraint enforced)
phone	VARCHAR(20)	Ethiopian phone number
password	VARCHAR(255)	BCrypt hashed password
city	VARCHAR(100)	Ethiopian city (default: Addis Ababa)
address	TEXT	Delivery address
role	ENUM('customer','admin')	Access level
is_active	TINYINT(1)	Soft delete flag
created_at / updated_at	TIMESTAMP	Audit timestamps

Table: categories

Product categories with slugs used in URL filtering.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Category ID
name	VARCHAR(100)	Display name
slug	VARCHAR(100) UNIQUE	URL-friendly identifier
icon	VARCHAR(20)	Icon reference
sort_order	INT	Display ordering

Table: products

Core product catalog. References categories via a foreign key.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Product ID
category_id	INT FK → categories.id	Category (CASCADE DELETE)

Column	Type	Description
name	VARCHAR(200)	Product name
description	TEXT	Full description
price	DECIMAL(10,2)	Current price in ETB
original_price	DECIMAL(10,2)	Strikethrough price for discounts
stock	INT	Available quantity
image_url	VARCHAR(500)	External image URL
badge	ENUM('new','sale','hot','')	Promotional badge
rating	DECIMAL(3,1)	Average review rating
review_count	INT	Number of reviews
is_active	TINYINT(1)	Visibility flag

Table: cart_items

Stores a user's active cart. A UNIQUE KEY on (user_id, product_id) prevents duplicate entries — quantity is incremented instead.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Cart item ID
user_id	INT FK → users.id	Owning user (CASCADE)
product_id	INT FK → products.id	Product reference (CASCADE)
quantity	INT	Item quantity

Table: orders

Order headers. Each order has a unique human-readable order number (e.g. HM-A3F9B1C2).

Column	Type	Description
id	INT AUTO_INCREMENT PK	Order ID
user_id	INT FK → users.id	Customer (CASCADE)
order_number	VARCHAR(30) UNIQUE	Formatted order reference
total_amount	DECIMAL(10,2)	Grand total including shipping
shipping_address	TEXT	Delivery address
city	VARCHAR(100)	Delivery city

Column	Type	Description
phone	VARCHAR(20)	Contact phone
payment_method	ENUM(...)	cash_on_delivery / bank_transfer / mobile_money
status	ENUM(...)	pending / processing / shipped / delivered / cancelled
notes	TEXT	Optional delivery notes

Table: order_items

Line items for each order. Stores a snapshot of product_name and price at the time of purchase (important for price history).

Column	Type	Description
id	INT AUTO_INCREMENT PK	Line item ID
order_id	INT FK → orders.id	Parent order (CASCADE)
product_id	INT FK → products.id	Product reference (CASCADE)
product_name	VARCHAR(200)	Name snapshot at purchase
price	DECIMAL(10,2)	Price snapshot at purchase
quantity	INT	Ordered quantity

Table: reviews

One review per user per product (enforced by UNIQUE KEY on (product_id, user_id)). After each review submission, the product's average rating and review_count are recalculated and saved to the products table.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Review ID
product_id	INT FK → products.id	Reviewed product
user_id	INT FK → users.id	Reviewer
rating	TINYINT (1–5)	Star rating with CHECK constraint
comment	TEXT	Written review

Table Relationships Summary

- categories → products: One-to-Many (one category has many products)
 - users → cart_items: One-to-Many (one user has many cart items)
 - users → orders: One-to-Many (one user places many orders)
 - orders → order_items: One-to-Many (one order has many line items)
 - products → order_items: One-to-Many (one product appears in many orders)
 - products ↔ users via reviews: Many-to-Many
-

4. File Structure

```
ethio-market/
├── index.php           # Home page
├── products.php        # Product listing with filters
├── product_detail.php  # Single product view + reviews
├── cart.php            # Shopping cart
├── checkout.php        # Checkout form
├── order_success.php   # Order confirmation page
├── orders.php          # Customer order history
├── profile.php         # User profile management
├── login.php           # Login form
├── signup.php          # Registration form
├── logout.php          # Session destruction
├── connection.php      # Database connection (env-aware)
├── router.php          # URL routing helper
├── includes/
│   ├── header.php      # HTML head + navbar (shared)
│   └── footer.php       # Footer + script tags (shared)
├── actions/
│   ├── login_action.php # POST handler: authenticate user
│   ├── signup_action.php # POST handler: register user
│   ├── checkout_action.php # POST handler: create order
│   ├── cart_add.php      # Redirect-based cart add
│   └── cart_remove.php   # Redirect-based cart remove
├── ajax/
│   ├── search.php       # AJAX: live product search (JSON)
│   ├── cart_action.php  # AJAX: add/update/remove cart (JSON)
│   └── validate_email.php # AJAX: check email availability (JSON)
├── admin/
│   ├── index.php        # Admin dashboard with stats
│   ├── products.php     # Product list with edit/delete
│   ├── add_product.php  # Add new product form
│   ├── edit_product.php # Edit existing product form
│   ├── orders.php       # Order management with status update
│   ├── users.php        # User list with order stats
│   └── includes/
```

├──	admin_header.php	# Admin HTML head + sidebar nav
├──	admin_footer.php	# Admin closing tags
├──	actions/	
│ ├──	save_product.php	# POST handler: create/update product
│ └──	delete_product.php	# GET handler: soft-delete product
├──	css/	
│ └──	style.css	# All styles (1500+ lines, CSS custom properties)
├──	js/	
│ └──	main.js	# All client-side JavaScript
├──	database/	
│ └──	habesha_market.sql	# Full schema + sample data

5. Page-by-Page Code Analysis

5.1 Shared Components

connection.php

This file establishes the MySQL connection. It reads credentials from environment variables first, falling back to hardcoded defaults. This pattern supports both local development and deployment on platforms like Replit that inject environment variables.

```
$con = mysqli_connect($dbHost, $dbUser, $dbPassword, $dbName, $dbPort, $dbSocket);
```

A fallback attempt without socket is made if the primary connection fails. utf8mb4 character set is applied on success, enabling full Unicode support for Amharic content.

includes/header.php

Included at the top of every user-facing page. Responsibilities:

- Starts the PHP session (session_start())
- Identifies the current page via basename(\$_SERVER['PHP_SELF']) for active nav highlighting
- Queries the cart item count for the logged-in user to display in the navbar badge
- Outputs the full HTML <head> section with meta tags, font imports (Poppins, Playfair Display), Font Awesome 6.5, and style.css
- Renders the responsive navbar with conditional links based on session state (guest vs. logged-in vs. admin)
- Renders the animated particle canvas element and the colour stripe below the navbar

The navbar has three distinct states: - **Guest:** Login and Sign Up links - **Logged-in Customer:** Cart (with badge), Orders, Profile, Logout - **Admin:** All customer links plus an Admin Panel button

includes/footer.php

Included at the bottom of every user-facing page. Outputs the full 4-column footer grid (brand info, quick links, account links, contact), the footer bottom row with Ethiopian flag colours, and the closing HTML. Also loads js/main.js before `</body>`.

5.2 Home Page — `index.php`

The homepage combines PHP data retrieval with structured HTML sections. Two database queries run on load:

- **Featured products:** 8 top-rated active products with JOIN to get category names
- **Categories:** All categories with a count of their active products (using COUNT + GROUP BY)

HTML Sections:

Section	Purpose
Hero	Full-viewport introductory section with animated image, floating info cards, and call-to-action buttons
Features Strip	5 USP cards: Fast Delivery, Secure Payments, Quality Assured, Local Artisans, Easy Returns
Categories	Responsive grid of category cards linking to filtered product listings
Featured Products	Product card grid with Add to Cart AJAX buttons
Counter Section	Animated number counters (triggered by Intersection Observer)
Testimonials	3 hardcoded customer review cards
Newsletter	A submit-only form that triggers a JS toast notification

The product cards on the homepage use data-id attributes and the class `.add-to-cart-ajax` so that `main.js` can intercept clicks and handle cart operations via the AJAX endpoint without a page reload.

5.3 Products Listing — `products.php`

This page handles full product browsing with three simultaneous filter mechanisms:

URL Parameters: - `?category=coffee` — filters by category slug - `?q=berbere` — full-text search across name, description, and category - `?sort=price_asc` — applies one of five sort orders

PHP Logic:

```
$where = ['p.is_active=1'];  
if ($category) $where[] = "c.slug='$category'";  
if ($search) $where[] = "(p.name LIKE '%$search%' OR p.description LIKE '%$search%' OR c.name  
LIKE '%$search%')";  
$where_sql = implode(' AND ', $where);
```

The \$order_map array maps URL sort values to SQL ORDER BY clauses.

Features: - Breadcrumb navigation that adapts to active category - Live search input that also works as a server-side search (two modes) - Category dropdown and sort dropdown — both use main.js event listeners to redirect on change - Checks which products the user already has in their cart to show a “View Cart” button instead of “Add” - Empty state display when no results found

5.4 Product Detail — product_detail.php

Accessed via ?id=N, this page loads a single product and handles multiple concerns:

Data Loaded: - Product + category JOIN - Check if the product is already in the current user’s cart - Last 10 reviews with user names - Up to 4 related products (same category, different product)

Review Submission:

Reviews are submitted via a standard POST form. On submission:

```
INSERT INTO reviews (product_id, user_id, rating, comment)  
VALUES ($id, $uid, $rating, '$comment')  
ON DUPLICATE KEY UPDATE rating=$rating, comment='$comment'
```

After insert, the product’s rating and review_count are recalculated:

```
UPDATE products SET rating=$avg, review_count=$cnt WHERE id=$id
```

Interactive Star Rating:

An inline JavaScript block (highlightStars, resetStars, selectStar) handles the visual star rating interaction. The selected value is stored in a hidden input field.

Actions: - If in stock and not in cart: “Add to Cart” AJAX button - If in stock and already in cart: “View in Cart” redirect button - If out of stock: Error message, no purchase options

5.5 Shopping Cart — cart.php

Requires login (redirects to login.php if no session). Retrieves all cart items for the user with a JOIN across cart_items, products, and categories.

Layout:

Two-column grid: cart item table on the left, order summary on the right (sticky on desktop).

Quantity Controls:

Each item has plus/minus buttons with data-action and data-id attributes. These trigger main.js event listeners that POST to ajax/cart_action.php. On success, the quantity, item subtotal, and cart total are updated in the DOM without a page reload.

Remove Item:

Uses a redirect link to actions/cart_remove.php?id=N with a JavaScript confirm() dialog.

Pricing Logic:

- Subtotal: sum of price × quantity for all items
 - Shipping: flat ETB 50 (free if cart is empty)
 - Total: subtotal + shipping
 - Tax is included (shown as “Included” in UI)
-

5.6 Checkout — checkout.php

Requires login and a non-empty cart (redirects to cart.php otherwise). Pre-fills form fields from the user’s profile.

Form Fields: - Full Name (pre-filled) - Phone Number (pre-filled) - City (dropdown of major Ethiopian cities) - Delivery Address (pre-filled) - Order Notes (optional) - Payment Method (radio buttons: Cash on Delivery, Bank Transfer, Mobile Money)

Order Summary Panel:

Shows each cart item image, name, quantity, and line total. Also displays subtotal, shipping, and grand total. Sticky positioned on desktop.

The form posts to actions/checkout_action.php. Errors are stored in \$_SESSION['checkout_error'] and displayed on redirect back.

5.7 Authentication — login.php / signup.php

Both pages redirect to index.php if the user is already logged in.

login.php:

A minimal centred card form. Displays error messages from \$_SESSION['login_error'] and success messages from \$_SESSION['signup_success']. Client-side validation (email format, minimum password length) runs before submission. Actual authentication happens in actions/login_action.php.

signup.php:

A wider form card with a two-column grid layout. Fields include full name, email, phone, city (dropdown), address, password, and confirm password. Features:

- Real-time email availability check via ajax/validate_email.php (triggered after 600ms debounce)
- All fields validated client-side before submission
- Terms and Conditions checkbox (required)

actions/login_action.php:

Queries the database for the user by email, then verifies the password using password_verify(). On success, stores user data in session. Redirects to index.php on success or back to login.php on failure.

actions/signup_action.php:

Validates all inputs server-side, checks for existing email, then inserts the new user with a bcrypt-hashed password using password_hash(\$password, PASSWORD_DEFAULT).

5.8 User Profile — profile.php

Requires login. Loads user data and shows two forms: Profile Update and Change Password.

Profile sidebar shows: - Initial avatar (first letter of name) - Name and email - Order count and cart item count - Navigation links to Profile, Orders, Cart, Logout

Profile Update form updates full_name, phone, city, and address. Email is shown as read-only.

Change Password form verifies current password with password_verify(), then hashes and saves the new password with password_hash().

Both forms redirect back to profile.php with success/error messages in session.

5.9 Orders — orders.php / order_success.php

orders.php: Lists all orders for the logged-in user in reverse chronological order. Each order card shows the order number, date, status badge, delivery city, payment method, line items, and total. Status badges use colour-coded CSS classes: pending (cyan), processing (blue), delivered (green), cancelled (red).

order_success.php: Shown after a successful checkout. Reads \$_SESSION['order_success'] for the order number, then clears the session key. Displays a confirmation card with order number, estimated delivery, and action buttons. Redirects to orders.php if accessed without a valid session key.

5.10 Admin Panel

All admin pages are protected by `admin_header.php` which checks:

```
if (!isset($_SESSION['user_id']) || ($_SESSION['role'] ?? '') !== 'admin') {  
    header('Location: ../login.php');  
    exit;  
}
```

The admin layout uses a two-column CSS Grid: a 240px sticky sidebar and a main content area. On screens below 1024px, the sidebar is hidden.

admin/index.php — Dashboard:

Runs 4 aggregate queries to display stat cards: Active Products, Total Orders, Customers, Total Revenue. Shows a Recent Orders table (last 8) and a Low Stock Alert panel (products with stock ≤ 5).

admin/products.php — Product List:

Full table of all active products with image thumbnails, name, category, price (with strikethrough original), stock (colour coded: green/yellow/red), badge, and rating. Supports category filter dropdown. Each row has Edit and Delete action buttons.

admin/add_product.php — Add Product:

A form-card with a two-column grid. Fields: name, category, badge, price, original price, stock, image URL, and description. An image preview updates live as the URL input changes (JavaScript `updatePreview()` function). Posts to `actions/save_product.php` with `action=add`.

admin/edit_product.php — Edit Product:

Same form as Add Product but pre-populated with existing product data. Posts to `actions/save_product.php` with `action=edit`.

admin/orders.php — Order Management:

Full order table with customer name, email, amount, payment method, status, and date. Admins can update order status via an inline `<select>` dropdown that navigates to `?id=N&set_status=VALUE` on change.

admin/users.php — User List:

Shows all registered users with their order count and total lifetime spend (calculated via `LEFT JOIN + SUM` aggregate query). Role is shown as a colour-coded badge.

6. CSS Design System and Styling Conventions

The entire visual design is defined in `css/style.css` (approximately 1,500 lines). It uses CSS Custom Properties (variables) for a consistent theme, making future reskinning straightforward.

Design Theme: Light Aqua

The theme is named “Light Aqua” and uses a clean, light background with aqua/teal primary colours.

CSS Custom Properties (Root Variables)

```
:root {
  --primary:      #0db2cf; /* Main aqua/teal */
  --primary-dark: #078ca7; /* Darker teal for gradients */
  --primary-light: #56d8e8; /* Lighter teal for text accents */
  --secondary:    #14c7b5; /* Green-teal for ratings, prices */
  --accent:       #f97316; /* Orange for "Hot" badges, errors */
  --accent-dark:  #ea580c; /* Darker orange */
  --dark:         #eef8fb; /* Page background light blue-white */
  --dark2:        #ffffff; /* Pure white for cards */
  --card-bg:      rgba(255,255,255,0.88);
  --glass:        rgba(255,255,255,0.78); /* Glassmorphism card bg */
  --glass-border: rgba(13,178,207,0.2); /* Subtle aqua border */
  --text-light:   #123847; /* Dark blue-grey for body text */
  --text-muted:   #5c7885; /* Lighter text for secondary info */
  --shadow:       0 20px 60px rgba(12,56,72,0.16);
  --shadow-sm:    0 8px 30px rgba(12,56,72,0.12);
  --radius:       18px; /* Large card border radius */
  --radius-sm:    10px; /* Smaller elements */
  --transition:   all 0.35s cubic-bezier(0.4,0,0.2,1);
}
```

Typography

Font	Usage
Poppins (Google Fonts)	Body, navigation, labels, buttons — weights 300–900
Playfair Display (Google Fonts)	Section titles, hero title, form titles — weights 700, 800

Key UI Patterns

Glassmorphism Cards: Used throughout for product cards, form cards, cart summary, and admin stat cards. Applied via background: var(--glass) and backdrop-filter: blur(20px).

Gradient Buttons: .btn-primary-custom uses linear-gradient(135deg, var(--primary-dark), var(--primary)) with a pseudo-element overlay for hover reversal. Lift effect on hover via transform: translateY(-3px).

Particle Canvas: A full-screen fixed <canvas> element renders 65 floating particles in aqua/teal colours. Implemented in main.js using the Canvas 2D API with requestAnimationFrame.

Scroll Reveal Animations: Elements with .animate-in class are observed by an IntersectionObserver. When 10% visible, visible class is added which transitions opacity from 0 to 1 and transform: translateY(40px) to translateY(0). Delay utilities .delay-1 through .delay-5 stagger sibling elements.

Responsive Design — Breakpoints:

Breakpoint	Behaviour
max-width: 1024px	Hero goes single-column, hero visual hidden, cart/checkout/profile/admin layouts stack vertically
max-width: 768px	Hamburger menu appears, desktop nav hidden, footer goes single-column, cart table headers hidden
max-width: 480px	Form card padding reduced

Component Reference

Class	Element	Description
.navbar-custom	<nav>	Fixed top navbar with backdrop blur
.btn-primary-custom	<a> / <button>	Primary action button with gradient
.btn-outline-custom	<a> / <button>	Ghost button with border
.form-card	<div>	Centred glass card for forms
.form-control-custom	<input> / <select> / <textarea>	Styled input with focus glow
.product-card	<div>	Glass card for products with hover lift
.category-card	<a>	Category navigation card
.cart-summary	<div>	Sticky order summary panel
.admin-layout	<div>	Two-column grid: sidebar + main
.alert-custom	<div>	Contextual alert (error/success/info)
.order-status		Colour-coded order status badge
.animate-in	any	Scroll-reveal animation element
.toast-container	<div> (created by JS)	Fixed bottom-right toast stack

7. JavaScript Interactivity — main.js

All JavaScript is contained in js/main.js and runs inside a single DOMContentLoaded listener.

Navbar Scroll

Adds .scrolled class to #main-navbar when the user scrolls more than 50px. This increases background opacity and adds a box-shadow for visual depth.

Hamburger Menu

Toggles .open on the #nav-links element on hamburger click. Closes automatically on outside click.

Particle Animation

Creates 65 particles using the Canvas 2D API. Each particle has random position, velocity, radius, and colour from a palette of four aqua shades. The animation loop uses requestAnimationFrame. Particles bounce off canvas edges. Canvas resizes on window resize.

Scroll Reveal (IntersectionObserver)

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(e => {
    if (e.isIntersecting) {
      e.target.classList.add('visible');
      observer.unobserve(e.target);
    }
  });
}, { threshold: 0.1, rootMargin: '0px 0px -60px 0px' });
```

All .animate-in elements are observed. Once visible, they are unobserved to prevent repeated triggers.

Animated Counter

Counter elements with data-target attributes are observed by a second IntersectionObserver. When triggered, a setInterval increments a counter from 0 to the target value over approximately 2 seconds.

Live Search (AJAX)

Listens to the #live-search input. After 300ms debounce, issues a fetch call to ajax/search.php?q=.... The response JSON is mapped to HTML anchor elements and injected into #search-results-live. The dropdown closes on outside click.

Password Toggle

All .toggle-icon elements toggle the type attribute of their sibling input between password and text.

Signup Form Validation

The #signup-form runs a battery of client-side checks on submit: name length, email format, phone pattern, password length, and password match. Each field is marked with CSS .error or .success and a message is shown in the associated .field-msg element via setFieldState(). The email field also has an async availability check using ajax/validate_email.php after a 600ms debounce.

Login Form Validation

Validates email format and minimum password length before submission.

Cart Quantity Update (AJAX)

.qty-btn clicks send a POST to `ajax/cart_action.php` with `action=update`. On success, the quantity display, item subtotal, cart total, and navbar cart badge are all updated in-place.

Add to Cart (AJAX)

.add-to-cart-ajax clicks send a POST to `ajax/cart_action.php` with `action=add`. The button shows a spinner while waiting, then a checkmark on success. Unauthenticated users are redirected to `login.php`.

Toast Notifications

`window.showToast(msg, type)` creates a `div.toast-msg` in a fixed container. Toasts disappear after 3.5 seconds. Types: success, error, info.

Filter Dropdowns

`#category-filter` and `#sort-filter` changes update the page URL using `URL / URLSearchParams` and `navigate`.

8. AJAX Endpoints

All AJAX endpoints return `Content-Type: application/json`.

`ajax/search.php`

Method: GET | **Param:** q (string, min 2 chars)

Returns an array of up to 8 products matching the search query:

```
[{"id":1,"name":"Yirgacheffe Coffee","price":"450.00","image":"...", "category":"Ethiopian Coffee"}]
```

`ajax/cart_action.php`

Method: POST | **Param:** action (add/update/remove)

For add: `product_id` — inserts or increments cart item.

For update: `item_id`, `qty` — updates quantity.

For remove: `item_id` — deletes cart item.

Returns:

```
{"success":true,"count":3,"subtotal":900,"total":950}
```

Returns `{"success":false,"login":true}` for unauthenticated requests.

`ajax/validate_email.php`

Method: GET | **Param:** email

Returns:


```
{"taken": true}
```

9. Security Implementation

Concern	Implementation
Password Storage	password_hash() with PASSWORD_DEFAULT (bcrypt)
Password Verification	password_verify() — timing-safe comparison
SQL Injection	mysqli_real_escape_string() applied to all user inputs before SQL insertion
Session Management	session_start() used consistently; session destroyed completely on logout (session array cleared, cookie expired, session_destroy())
Authentication Guards	All authenticated pages check isset(\$_SESSION['user_id']) at the top
Admin Authorization	Admin pages check \$_SESSION['role'] === 'admin'
Input Sanitization	htmlspecialchars() on all output to prevent XSS
Email Validation	filter_var(\$email, FILTER_VALIDATE_EMAIL) on server side
CSRF Awareness	Payment method validated against an allowlist
Stock Control	AJAX cart endpoint checks product stock before adding
Integer Casting	All ID values cast to (int) before SQL use

10. Setup Instructions

Requirements

- PHP 7.4 or higher with mysqli extension enabled
- MySQL 5.7 or higher
- A web server (Apache/Nginx) or PHP built-in server

Step 1 — Database Setup

Open your MySQL client and run:

```
SOURCE /path/to/ethio-market/database/habesha_market.sql;
```

This creates the database, all tables, and inserts sample data including categories, 20 products, and 3 user accounts.

Step 2 — Configure Connection

Open connection.php and verify or update the defaults:

```
$dbHost = getenv('DB_HOST') ?: '127.0.0.1';
$dbUser = getenv('DB_USER')  ?: 'habesha_market_user';
$dbPassword = getenv('DB_PASSWORD') ?: 'habesha_market_password';
$dbName = getenv('DB_NAME')  ?: 'habesha_market';
$dbPort = (int)(getenv('DB_PORT') ?: 3306);
```

Alternatively, set environment variables in a .env file or your server configuration.

Step 3 — Create Database User (if needed)

```
CREATE USER 'habesha_market_user'@'localhost' IDENTIFIED BY 'habesha_market_password';
GRANT ALL PRIVILEGES ON habesha_market.* TO 'habesha_market_user'@'localhost';
FLUSH PRIVILEGES;
```

Step 4 — Start the Server

Using PHP built-in server:

```
cd /path/to/ethio-market
php -S localhost:8000 router.php
```

Or configure a virtual host in Apache/Nginx pointing to the project root.

Step 5 — Access the Application

- **Storefront:** http://localhost:8000
- **Admin Panel:** http://localhost:8000/admin/

Demo Accounts

Role	Email	Password
Admin	admin@habeshamarket.et	password
Customer	abebe@example.com	password
Customer	tigist@example.com	password

11. Features Implemented

Customer-Facing Features

- User registration with real-time email availability check (AJAX)
- User login with bcrypt password verification
- Full session management (login, logout, persistent auth)
- Browse all products with responsive product card grid
- Filter products by category (URL parameter)
- Live product search (AJAX dropdown, debounced 300ms)
- Server-side search with text-match across name, description, category

- Sort products by Featured, Newest, Price Low-High, Price High-Low, Top Rated
- Product detail page with image, description, stock status, pricing with discount calculation
- Star rating selector with hover preview
- Customer review submission (INSERT ON DUPLICATE KEY UPDATE)
- Related products section
- Add to Cart via AJAX (no page reload, spinner feedback)
- Cart quantity update via AJAX (subtotal and total update in DOM)
- Remove from cart
- Real-time cart badge counter in navbar
- Checkout form pre-populated from user profile
- Multiple payment methods (Cash on Delivery, Bank Transfer, Mobile Money)
- Order placement with automatic stock reduction
- Order confirmation page with order number
- Order history page
- Profile management (update info, change password)
- Toast notification system for user feedback
- Scroll reveal animations via IntersectionObserver
- Animated counting statistics
- Particle background animation

Admin Features

- Protected admin panel (role-based access control)
- Dashboard with live stats (products, orders, users, revenue)
- Recent orders table
- Low stock alert panel
- Product list with filter, thumbnails, and badge indicators
- Add product form with live image preview
- Edit product form
- Soft delete products
- Order management with inline status update dropdown
- User list with lifetime order value aggregation

12. Course Requirements Compliance

Requirement	Status	Implementation
Clean responsive UI	Met	CSS Grid/Flexbox, 3 responsive breakpoints
Mobile-friendly	Met	Hamburger menu, stacking layouts below 1024px
Client-side JS (form validation)	Met	Signup + login validation in main.js

Requirement	Status	Implementation
Client-side JS (DOM manipulation)	Met	Cart badge, quantity display, toast notifications
Asynchronous JS (AJAX/fetch)	Met	Live search, cart add/update, email validation
Server-side PHP (sessions)	Met	Login/logout, cart count, admin protection
Server-side PHP (request handling)	Met	All POST/GET actions in /actions and /ajax
Server-side PHP (business logic)	Met	Order creation, stock deduction, rating recalc
MySQL (minimum 3 tables)	Exceeded	6 tables implemented
MySQL (at least one relationship)	Exceeded	Multiple one-to-many and many-to-many relationships
CRUD operations	Met	Full CRUD on products, orders; CRU on users
SQL injection prevention	Met	mysqli_real_escape_string + integer casting
Input sanitization	Met	htmlspecialchars on all output
Password hashing	Met	password_hash() with PASSWORD_DEFAULT (bcrypt)
No mysql_* functions	Met	Only mysqli_* functions used
No Laravel/React/Vue	Met	Vanilla PHP, CSS, JS only
Minimum 5 database tables	Met	6 tables: users, categories, products, cart_items, orders, order_items, reviews
Source code (.html/.css/.js/.php/.sql)	Included	All files present in submission
Database export (.sql with sample data)	Included	database/habesha_market.sql
Project documentation	This document	System overview, ERD, screenshots guide, setup, features
Setup instructions	Included	Section 10 of this document

Documentation prepared for Habesha Market — Web Programming Course Project
Author: Group 9